PENETRATION TESTING REPORT

Kioptrix Level 3 (VM3) — Network & Web Application Penetration Test

From SQL Injection & Unauthenticated RCE to Full Root Compromise

Target System	Kioptrix Level 3 (VM3) — 192.168.56.106 / kioptrix3.com (192.168.1.15)
Attacker Host	Kali Linux — 192.168.56.102 / 192.168.1.13
Assessment Type	Black-box Network & Web Application Penetration Test
Environment	Isolated lab environment (VirtualBox host-only network)
Engagement	NTI Track 26 — Penetration Testing & Application Security, Final Project
Prepared By	Abdulrahman Wael Nabil & Abdulrahman Hossam Shaat
Classification	Confidential — Educational / Training Use Only

Prepared as the Final Project for the NTI Penetration Testing & Application Security Track

Date: August 2026

1. Executive Summary

This report documents a black-box penetration test performed against the Kioptrix Level 3 (VM3) virtual machine as the final project for the NTI Penetration Testing & Application Security track. The objective was to identify and exploit security weaknesses in the target's network services and web application, and to escalate access to the highest privilege level achievable.

The assessment began with network reconnaissance and enumeration, which identified a Gallarific/LotusCMS-powered web application. Two independent, chainable attack paths were identified: an unauthenticated remote code execution vulnerability in LotusCMS 3.0's eval() handling, and a SQL injection vulnerability in the gallery application's 'id' and 'sort' parameters. Both were exploited — the RCE via a public exploit script and a manual eval() proof-of-concept (phpinfo() and system command execution), and the SQL injection via sqlmap and manual UNION-based queries — to obtain a shell as the web server user (www-data) and to dump the application's credential database.

Local enumeration of the compromised host uncovered a company policy note instructing staff to use 'sudo ht' for file editing, and confirmed that the local user loneferret held a NOPASSWD sudo entitlement for the ht text editor. The cracked 'loneferret' credentials were reused over SSH, and the ht editor was then abused to directly rewrite /etc/sudoers, granting the account unrestricted root privileges and yielding a full root shell.

In total, the engagement demonstrates a complete attack chain — from unauthenticated web exploitation to full root compromise — driven by unsanitized SQL input, an unpatched CMS, plaintext and weakly-hashed credentials, credential reuse, and an unsafe sudo configuration. Section 6 summarizes each finding with its severity (ordered Critical → High → Medium → Low), and Section 8 provides remediation guidance for each issue.

2. Scope and Objectives

2.1 Scope

- Target host: Kioptrix Level 3 (VM3), reachable at 192.168.56.106 and, via a second network adapter and a /etc/hosts entry, at kioptrix3.com (192.168.1.15).
- All network services and the hosted Gallarific/LotusCMS web application were in scope.
- Testing was performed entirely within a private, isolated lab environment; no production or third-party systems were involved.

2.2 Objectives

- Enumerate all reachable services and web application functionality for exploitable weaknesses.
- Gain an initial foothold on the target host via any viable vulnerability.
- Escalate privileges to root and confirm full system compromise.
- Document every step, tool, and piece of evidence to produce a professional, reproducible report.

3. Methodology

The assessment followed a standard penetration testing methodology aligned with the Penetration Testing Execution Standard (PTES), consisting of the following phases:

- Reconnaissance & Information Gathering — identify live hosts and open services.
- Enumeration — fingerprint services, technologies, and hidden application content.

- Vulnerability Analysis — map discovered services/technologies and application parameters to known weaknesses.
- Exploitation — leverage identified vulnerabilities (SQL injection and RCE) to gain code execution and extract data.
- Post-Exploitation — enumerate the compromised host, harvest credentials, and identify escalation paths.
- Privilege Escalation — elevate access from a low-privileged user to root.
- Reporting — consolidate findings, evidence, severity ratings, and remediation guidance.

4. Tools Used

- Nmap — host discovery and service/version enumeration
- Gobuster — web content/directory discovery
- A public LotusCMS exploit script — remote code execution and reverse shell delivery
- Manual browser-based eval() injection — proof-of-concept RCE (phpinfo, system commands)
- sqlmap — automated SQL injection detection, database/table enumeration, and credential dumping
- Netcat — reverse shell listener
- phpMyAdmin / MySQL client — database access and credential verification
- sqlmap's built-in dictionary cracker & hashcat — offline password hash cracking
- SSH client — remote authentication with recovered credentials
- ht (HT Editor) — abused via sudo for privilege escalation

5. Technical Findings and Walkthrough

5.1 Reconnaissance

A ping sweep of the lab subnet identified live hosts, followed by a full TCP port/service scan against the target.

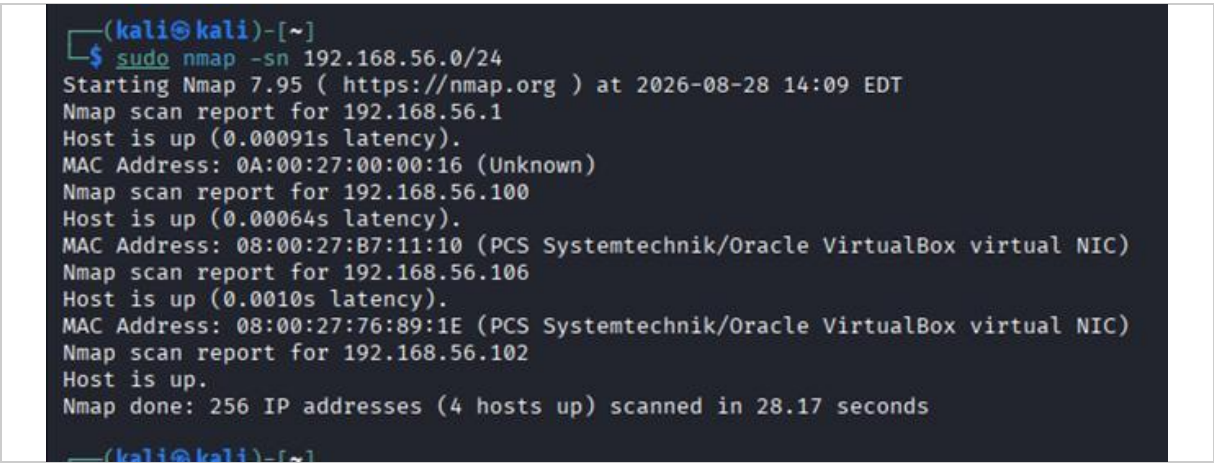


Figure 1. Host discovery sweep of 192.168.56.0/24 with Nmap (-sn).

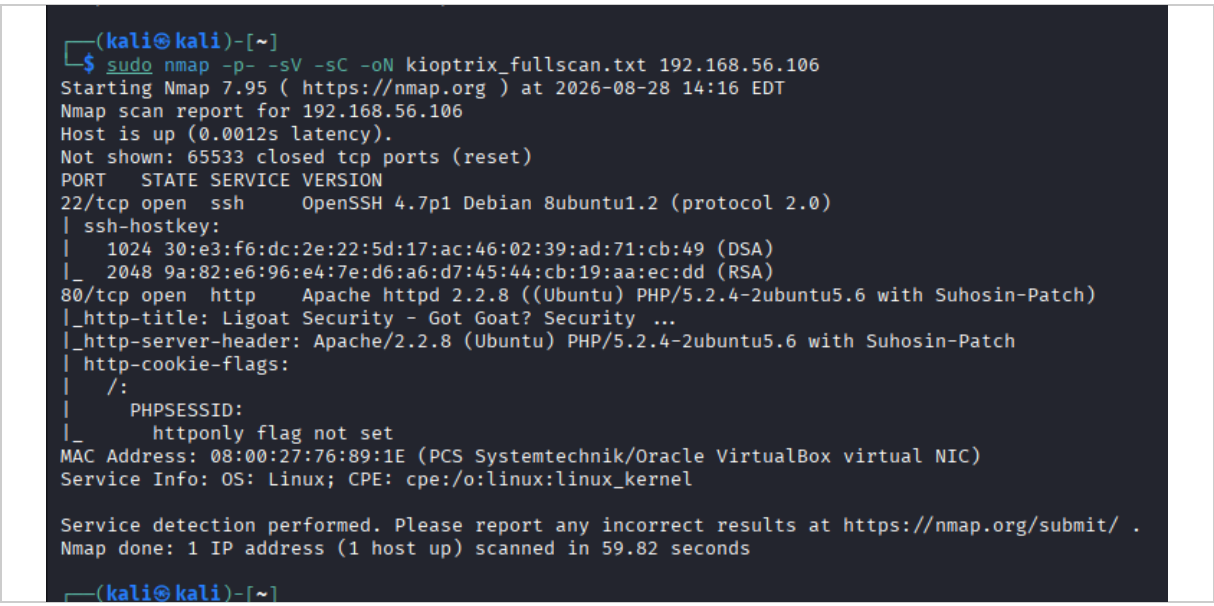


Figure 2. Full TCP port and service/version scan of 192.168.56.106 — SSH (22) and HTTP (Apache 2.2.8 / PHP 5.2.4) identified.

5.2 Enumeration

The web application was reviewed in the browser, its HTML source was inspected, and directory brute-forcing revealed hidden content, including a CMS administration login page.

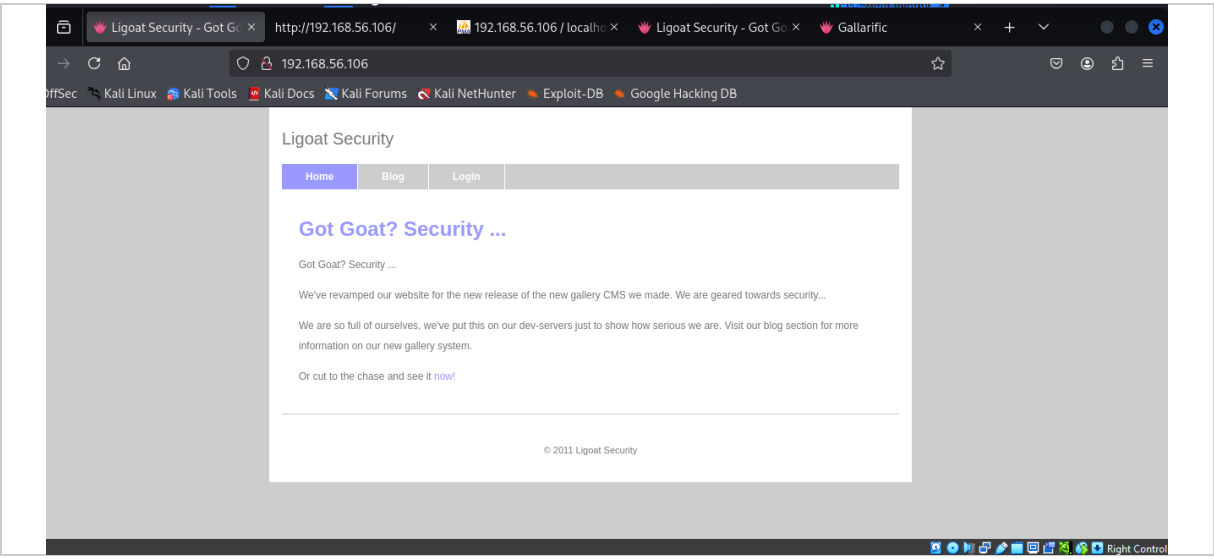


Figure 3. Target website homepage — 'Ligoat Security / Got Goat? Security' (Gallarific gallery CMS).

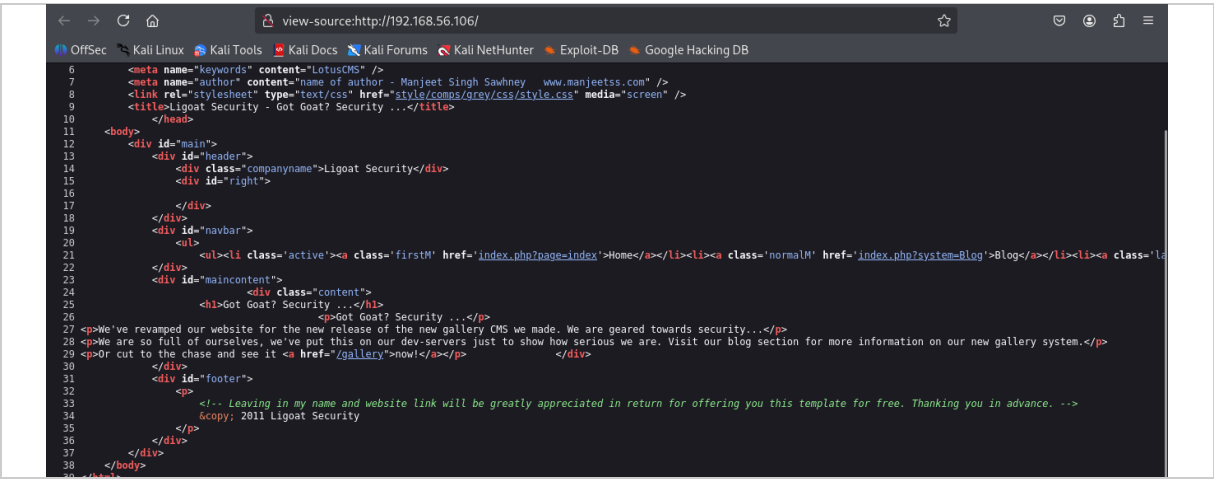


Figure 4. HTML source review revealing the LotusCMS-powered page structure.

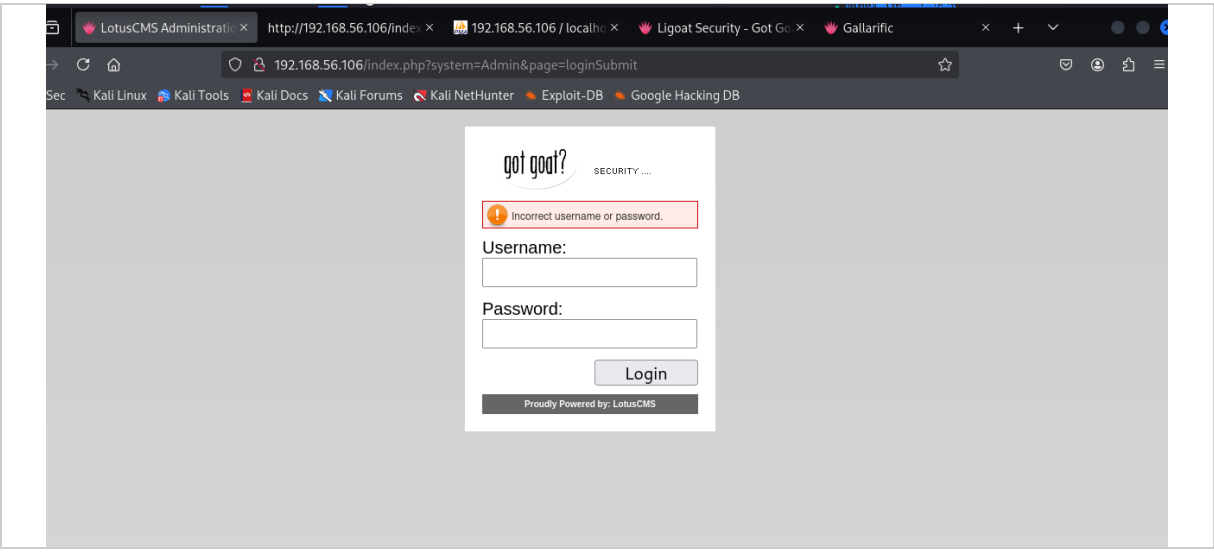


Figure 5. Discovered LotusCMS administration login page.

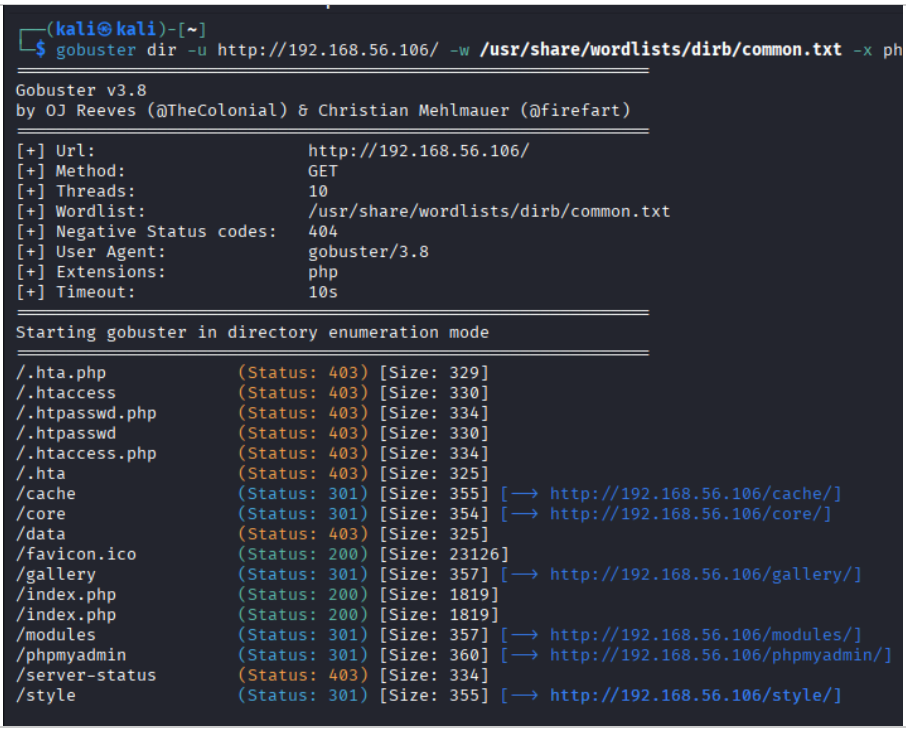


Figure 6. Gobuster directory enumeration exposing /cache, /core, /data, /gallery, /modules, /phpmyadmin, and /style.

5.3 Exploitation — LotusCMS Remote Code Execution

With LotusCMS 3.0 confirmed, a public exploit script was used to weaponize its known eval() code-injection weakness and deliver a reverse shell.



Figure 7. Launching the LotusCMS exploit script against the target.

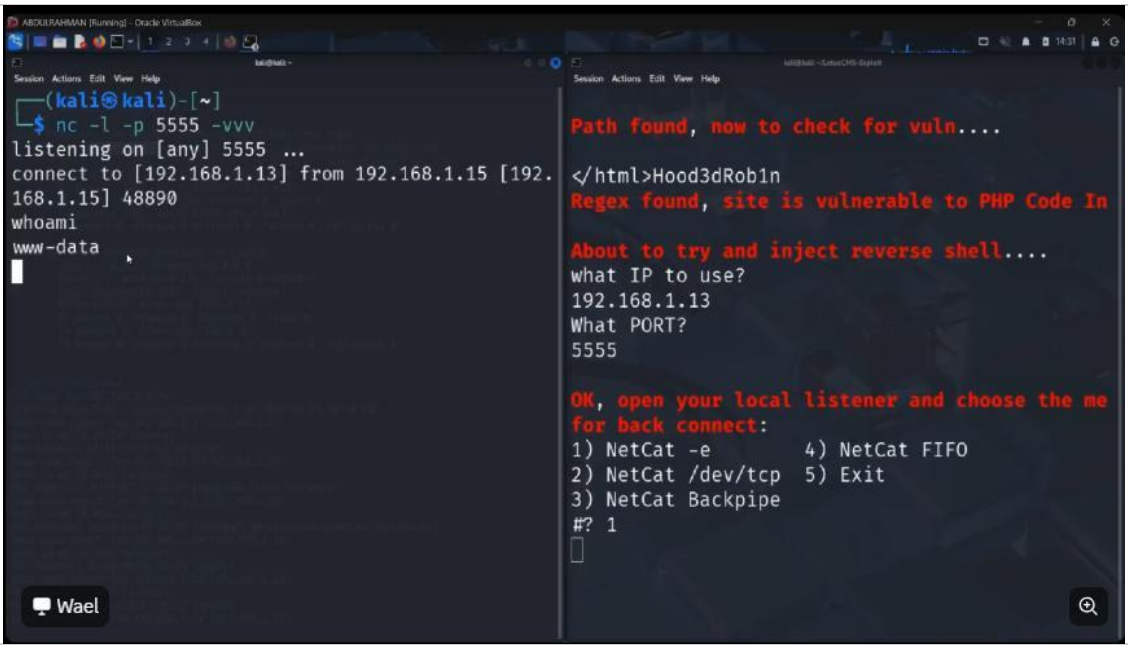


Figure 8. Script confirms the target is vulnerable to PHP code injection and a shell is caught (whoami = www-data).

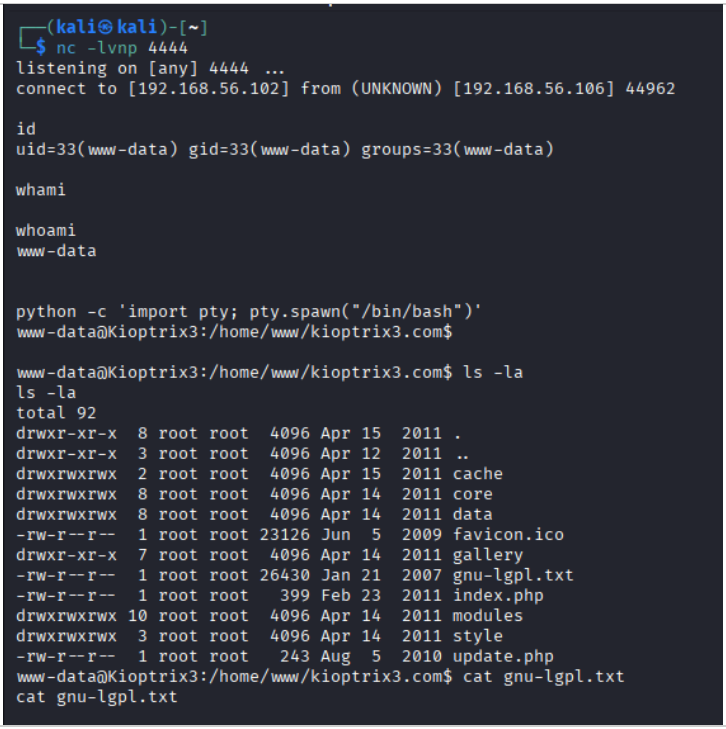


Figure 9. Reverse shell upgraded to an interactive TTY; id / whoami confirm www-data and the web root is listed.

5.4 Initial Post-Exploitation — Local Users & a Policy Clue

With code execution established, the host's user accounts and home directories were reviewed, surfacing a company policy document that hints at a privileged tool.

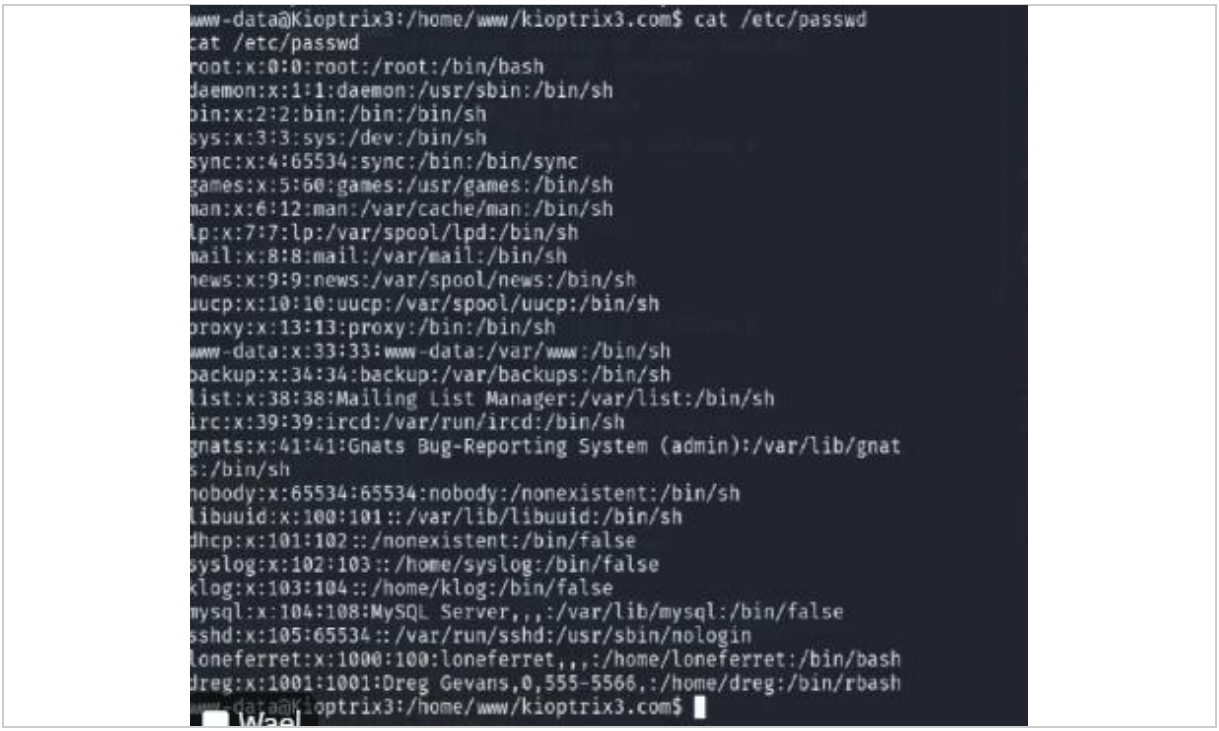


Figure 10. /etc/passwd reveals local accounts, including 'loneferret' and 'dreg'.



Figure 11. Close-up of the 'loneferret' and 'dreg' account entries.

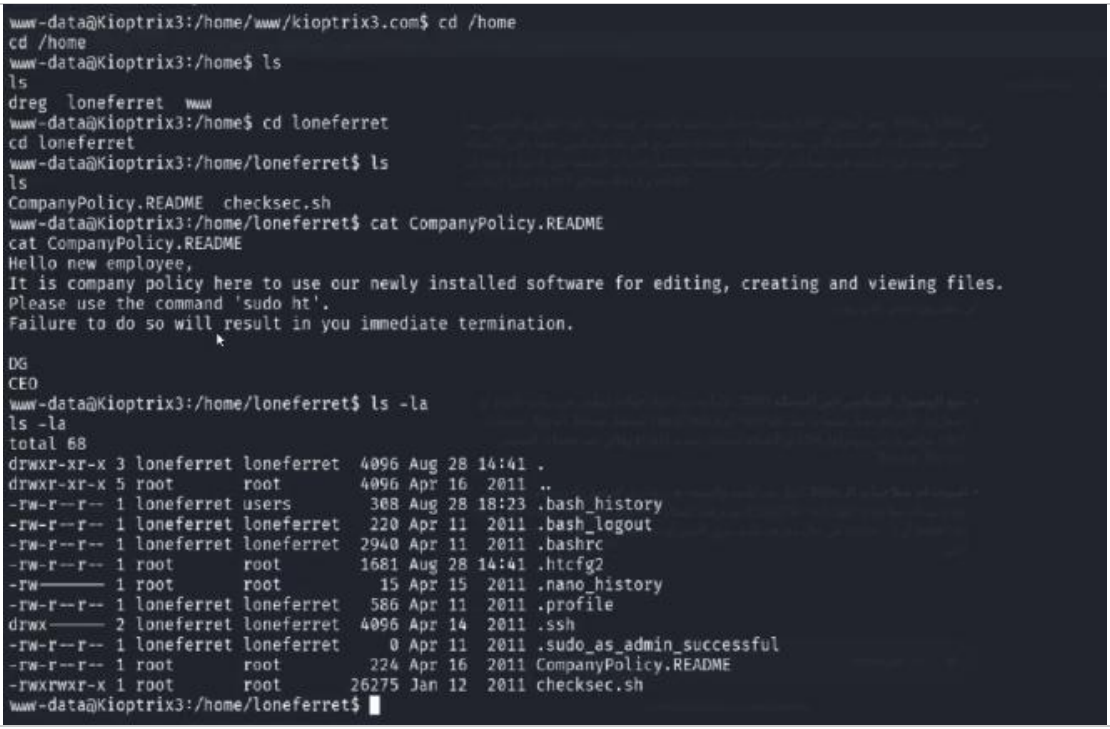


Figure 12. loneferret's home directory contains CompanyPolicy.README, instructing staff to use 'sudo ht' for file editing.



Figure 13. Attempting 'sudo ht' as www-data fails — the current shell user has no sudo rights; valid credentials are needed.

5.5 SQL Injection Discovery and Exploitation

The gallery application's photo-listing page was reviewed for injectable parameters. A local /etc/hosts entry was added to resolve the application's virtual host name, and sqlmap was used to confirm and exploit SQL injection in the 'id' parameter of gallery.php.

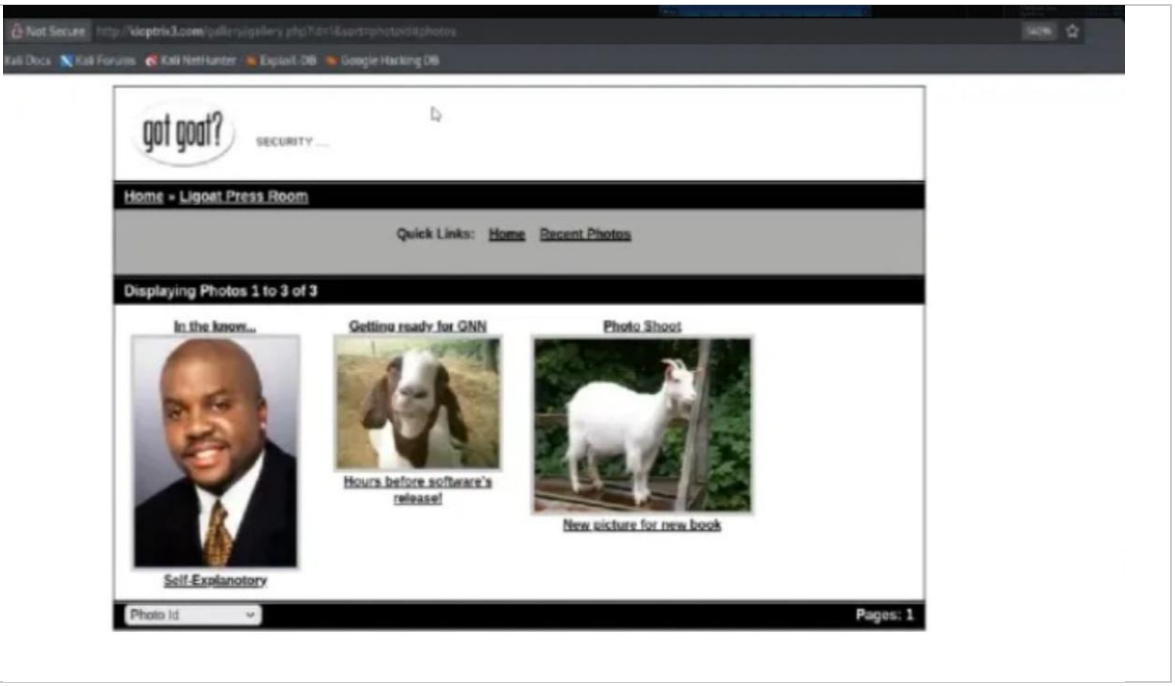


Figure 14. Gallery photo listing page — gallery.php?id=1&sort=photoid, the injectable endpoint.

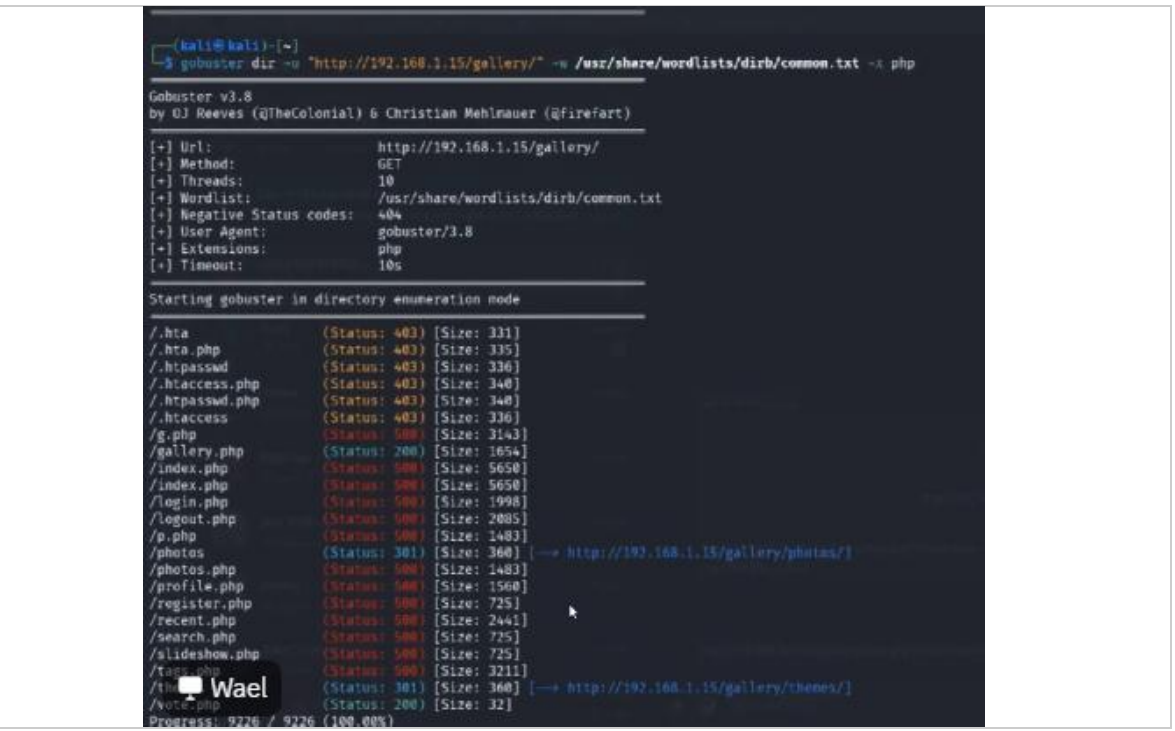


Figure 15. Directory enumeration of the /gallery application (via kioptrix3.com) reveals additional endpoints.

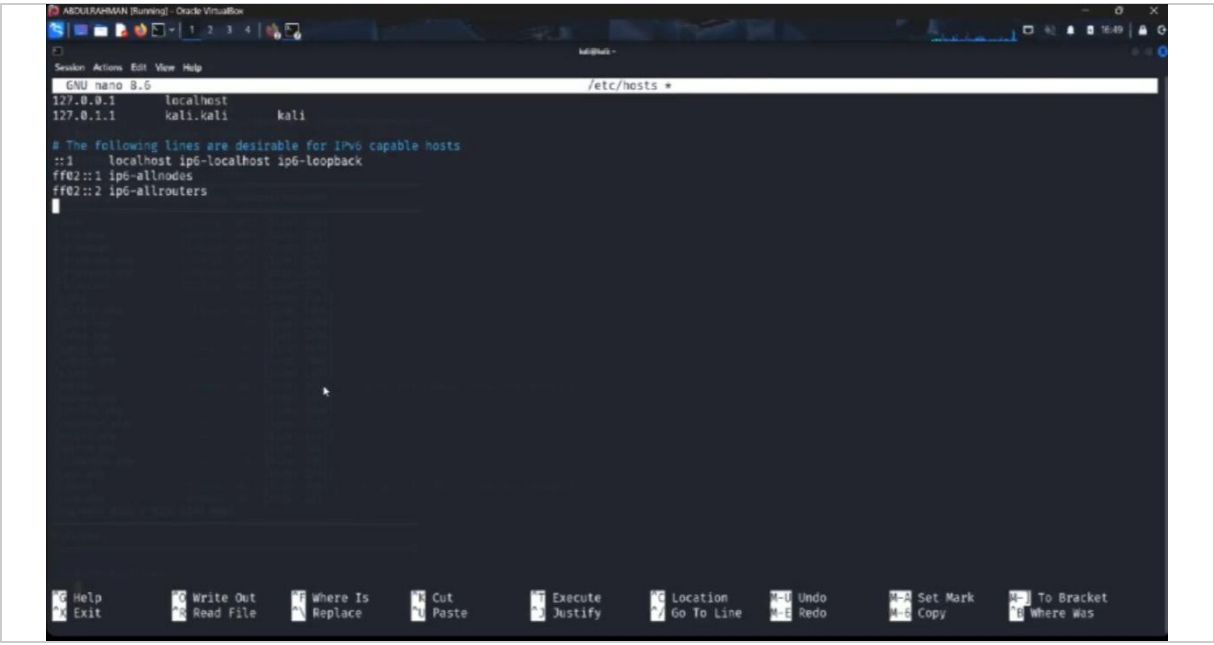


Figure 16. /etc/hosts prior to adding the target's virtual host entry.

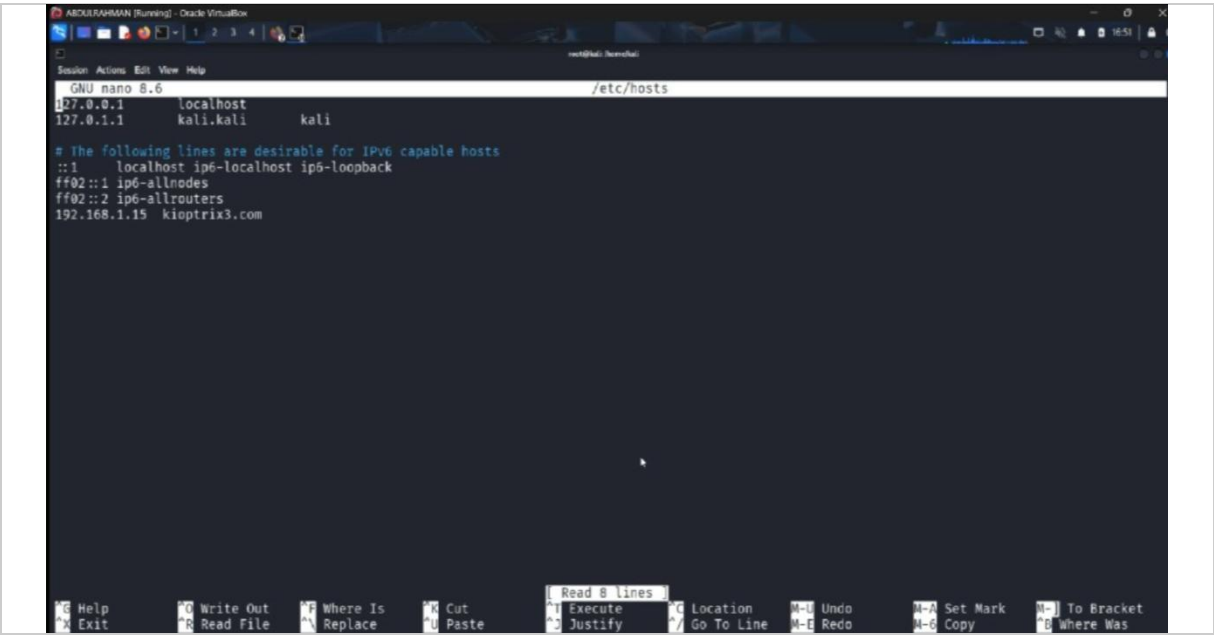


Figure 17. /etc/hosts updated to resolve kioptrix3.com to 192.168.1.15.



Figure 18. Launching sqlmap against gallery.php?id=1&sort=photoid to enumerate databases.

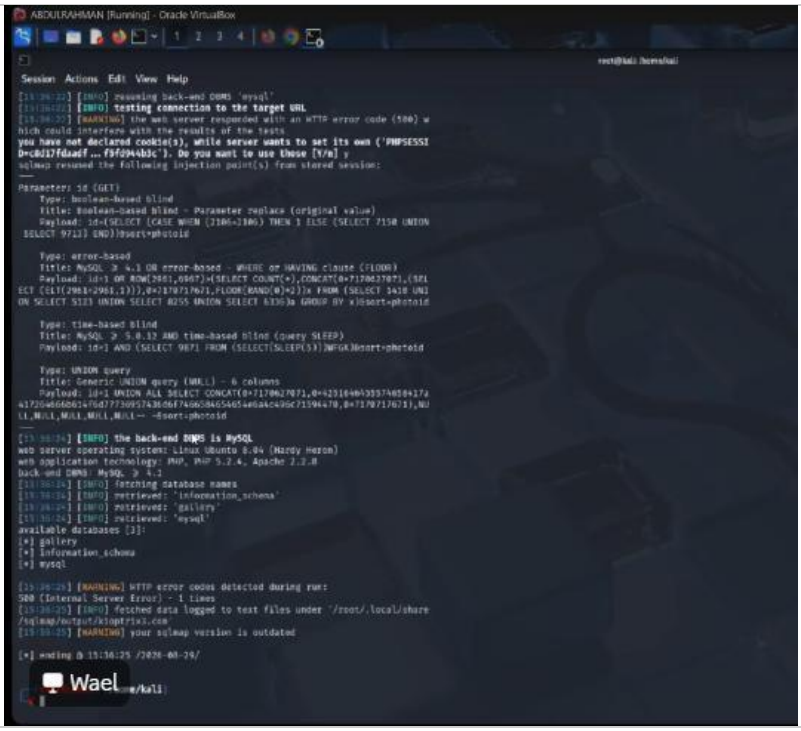


Figure 19. sqlmap confirms boolean-based, error-based, time-based, and UNION-based SQL injection; backend is MySQL; databases enumerated.



Figure 20. Enumerating tables inside the 'gallery' database.



Figure 21. Tables recovered, including the sensitive 'dev_accounts' table.



Figure 22. Dumping the 'dev_accounts' table with sqlmap.

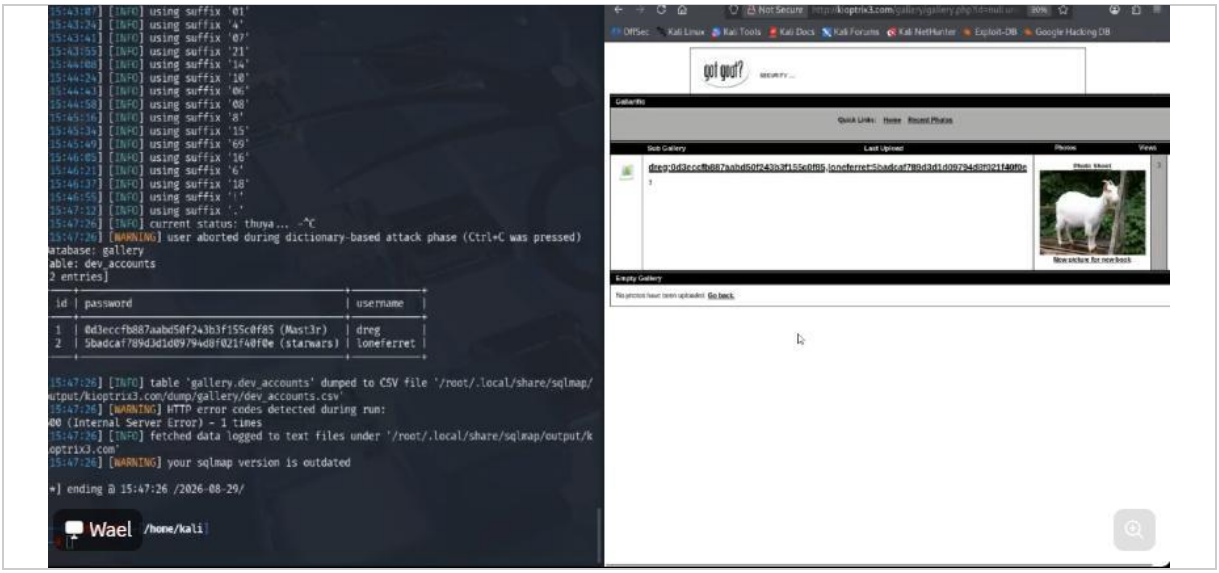


Figure 23. Credentials dumped and cracked (dreg = Mast3r, loneferret = starwars); the injected UNION payload is also reflected directly in the browser.

5.6 Confirming Code Execution via Manual eval() Injection

Independently of the exploit script, the LotusCMS eval() sink was probed manually through the browser to directly demonstrate arbitrary PHP code execution.

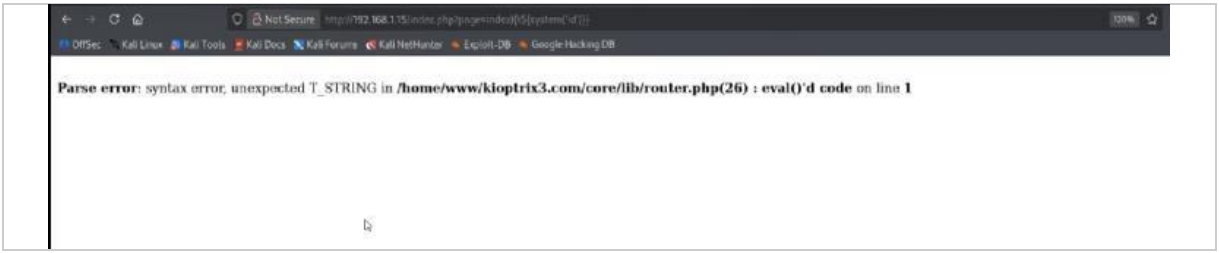


Figure 24. A malformed payload triggers a PHP parse error, disclosing the vulnerable eval()'d code path in core/lib/router.php.

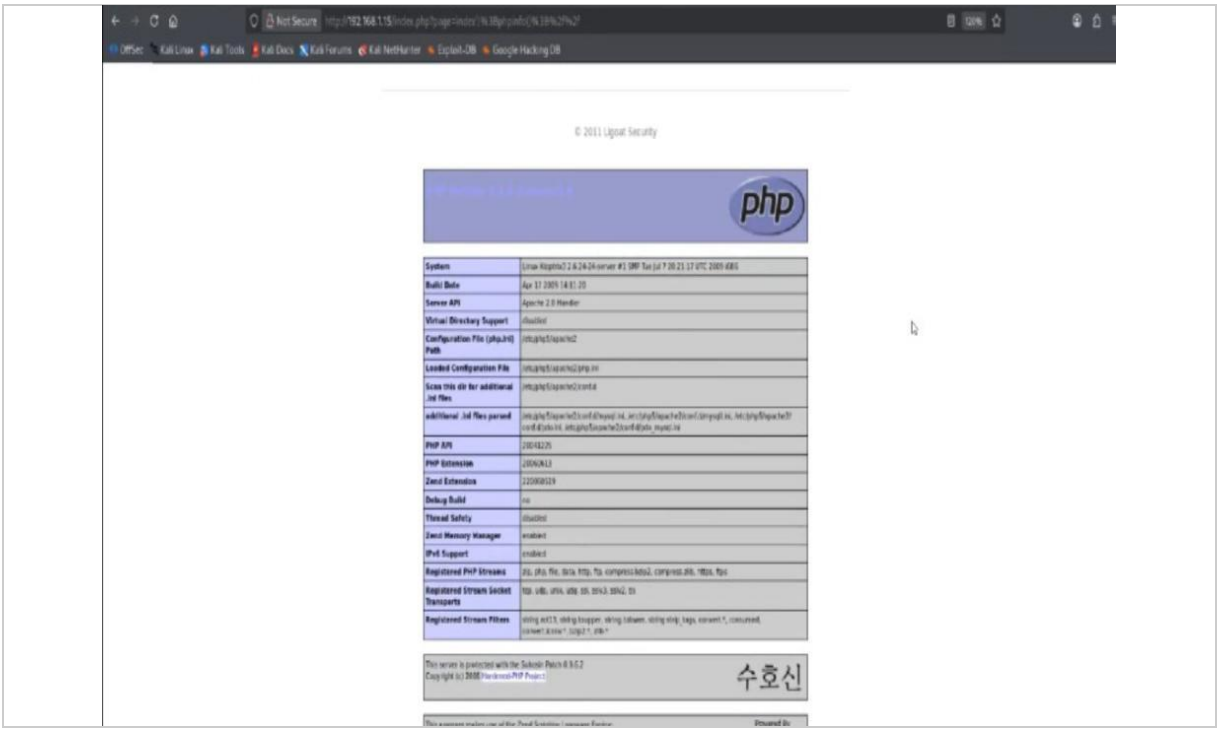


Figure 25. A corrected payload executes phpinfo(), proving arbitrary PHP code execution.

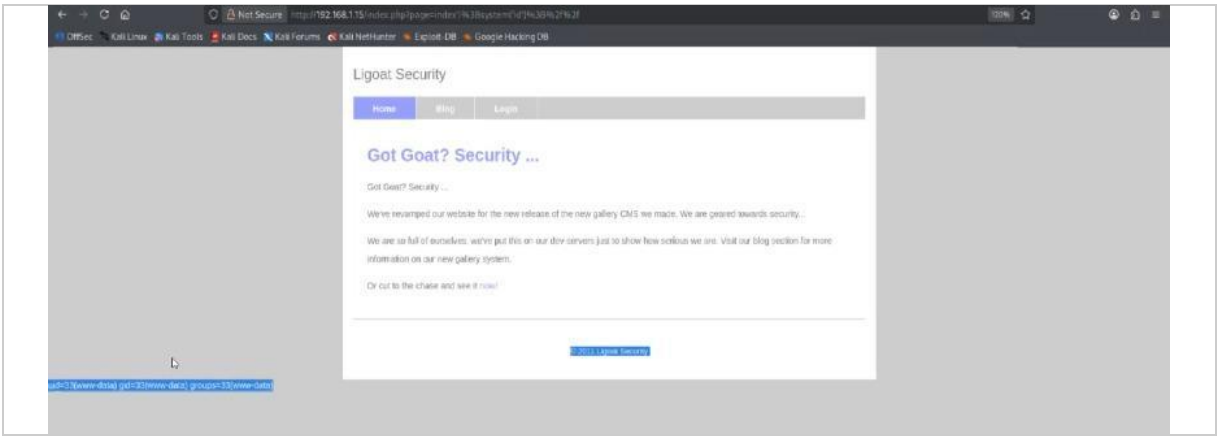


Figure 26. Executing the 'id' system command via the injection confirms OS command execution as www-data.

5.7 Further Local Enumeration and Credential Access

Local files were reviewed for additional credentials, and the database credentials recovered from the application configuration file were used to independently confirm the dumped accounts via direct MySQL access and phpMyAdmin.

```
www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$ cat admin.dat
cat admin.dat
|318d8dd409db395f0317efa71b3bad13e1fb9857|administrator|bla@bla.comwww-data@Kioptrix3:/home/www/kioptrix3.com/data/users$

www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$

www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$

www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$ uname -a
uname -a
Linux Kioptrix3 2.6.24-24-server #1 SMP Tue Jul 7 20:21:17 UTC 2009 i686 GNU/Linux
www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$

www-data@Kioptrix3:/home/www/kioptrix3.com/data/users$ find / -perm -4000 -type f 2>/dev/null
<w/kioptrix3.com/data/users$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/apache2/suexec
/usr/lib/pt_chown
/usr/bin/arping
/usr/bin/mtr
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/at
/usr/bin/sudoedit
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/traceroute6.iputils
/usr/local/bin/ht
```

Figure 27. admin.dat reveals an administrator MD5 hash; uname -a fingerprints the kernel; a SUID binary search begins.

```
www-data@Kioptrix3:/home/www/kioptrix3.com$ cat /home/www/kioptrix3.com/gallery/gconfig.php
<w/kioptrix3.com$ cat /home/www/kioptrix3.com/gallery/gconfig.php
<?php
        error_reporting(0);
        /*
                A sample Gallarific configuration file. You should edit
                the installer details below and save this file as gconfig.php
                Do not modify anything else if you don't know what it is.
        */

        // Installer Details ----------------------------------------

        // Enter the full HTTP path to your Gallarific folder below,
        // such as http://www.yoursite.com/gallery
        // Do NOT include a trailing forward slash

        $GLOBALS["gallarific_path"] = "http://kioptrix3.com/gallery";

        $GLOBALS["gallarific_mysql_server"] = "localhost";
        $GLOBALS["gallarific_mysql_database"] = "gallery";
        $GLOBALS["gallarific_mysql_username"] = "root";
        $GLOBALS["gallarific_mysql_password"] = "fuckeyou";

        // Setting Details ------------------------------------------

if(!$g_mysql_c = @mysql_connect($GLOBALS["gallarific_mysql_server"], $GLOBALS["gallarific_mysql_username"], $GLOBALS["gallarific_mysql_password"]
)) {
                echo("A connection to the database couldn't be established: " . mysql_error());
                die();
}else {
        if(!$g_mysql_d = @mysql_select_db($GLOBALS["gallarific_mysql_database"], $g_mysql_c)) {
                echo("The Gallarific database couldn't be opened: " . mysql_error());
                die();
        }else {
```

Figure 28. gallery/gconfig.php exposing MySQL root credentials in cleartext (root / fuckeyou).

```
        }
}

?>
www-data@Kioptrix3:/home/www/kioptrix3.com$ mysql -u root -pfuckeyou -e "SELECT * FROM gallery.dev_accounts;"
<root -pfuckeyou -e "SELECT * FROM gallery.dev_accounts;"
+----+----------+----------------------------------+
| id | username | password                         |
+----+----------+----------------------------------+
|  1 | dreg     | 0d3eccfb887aabd50f243b3f155c0f85 |
|  2 | loneferret | 5badcaf789d3d1d09794d8f021f40f0e |
+----+----------+----------------------------------+
www-data@Kioptrix3:/home/www/kioptrix3.com$
```

Figure 29. Using the recovered credentials to query gallery.dev_accounts directly from the shell — confirms the sqlmap dump.

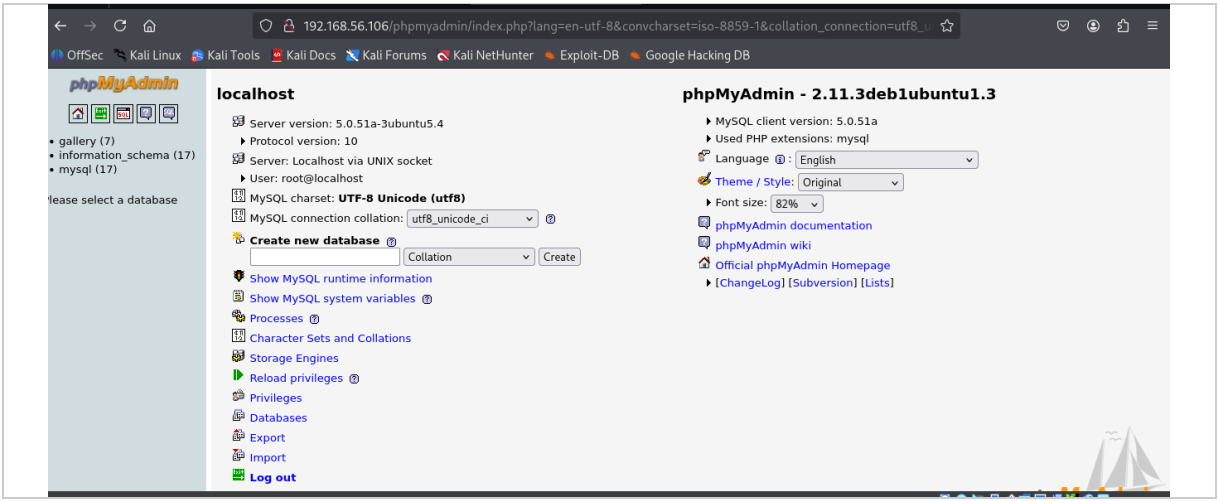


Figure 30. Authenticated access to phpMyAdmin using the recovered database credentials.

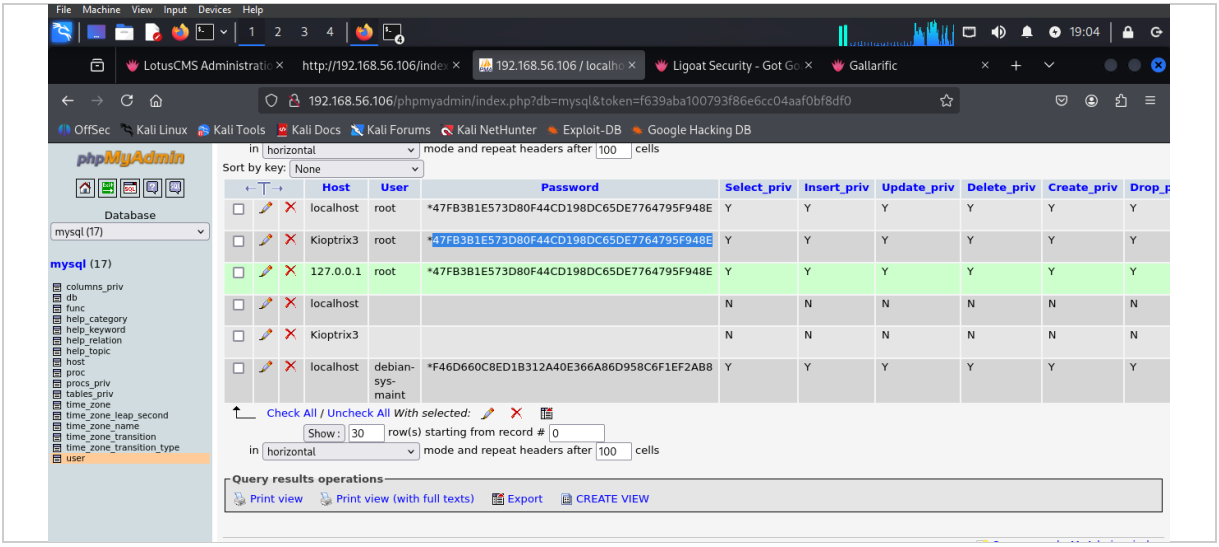


Figure 31. mysql.user table viewed through phpMyAdmin, exposing further password hashes.

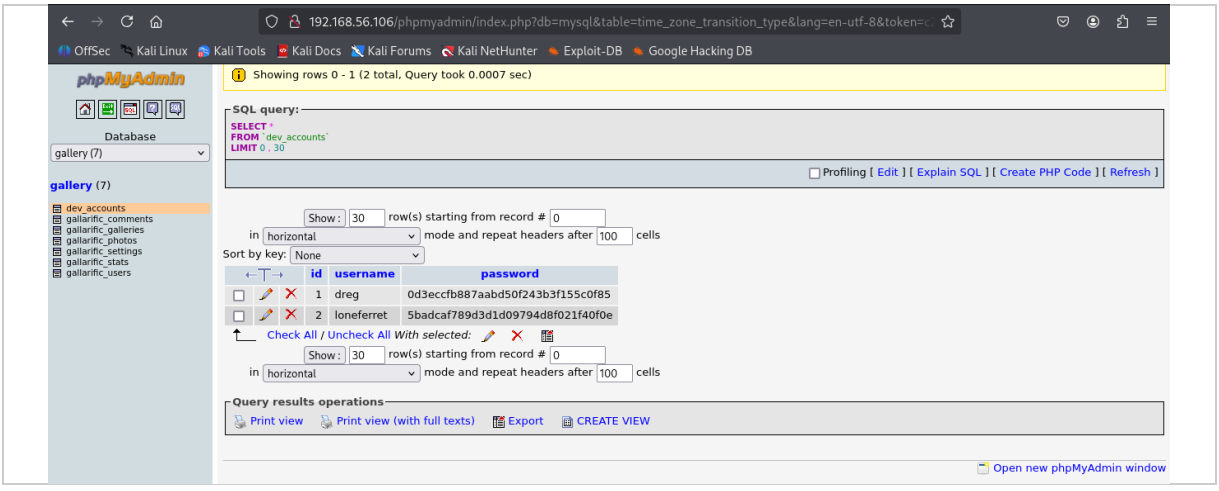


Figure 32. gallery.dev_accounts table confirmed through phpMyAdmin.

5.8 Privilege Escalation to Root

The cracked credentials for both local accounts were tested over SSH. 'dreg' had no sudo rights, but 'loneferret' was confirmed to hold a NOPASSWD sudo entitlement for the 'ht' editor — matching the hint found in CompanyPolicy.README. The editor was then abused to rewrite /etc/sudoers directly.

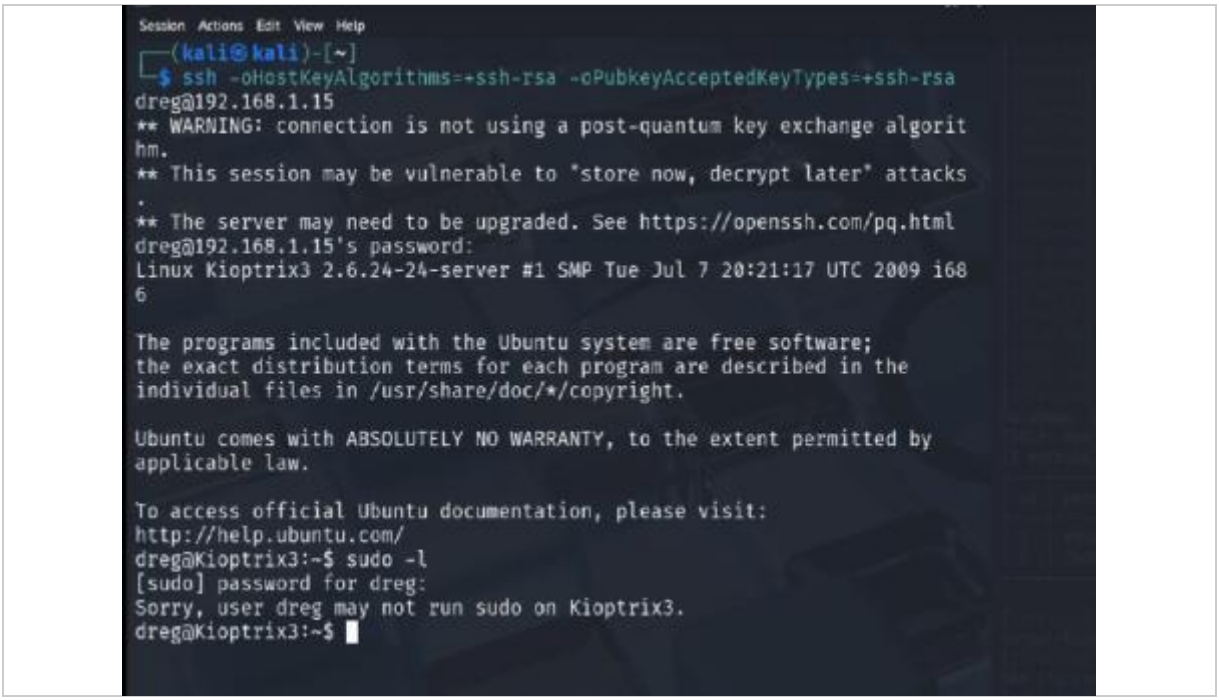


Figure 33. SSH login as 'dreg' (password: Mast3r); sudo -l confirms dreg has no sudo rights on the host.

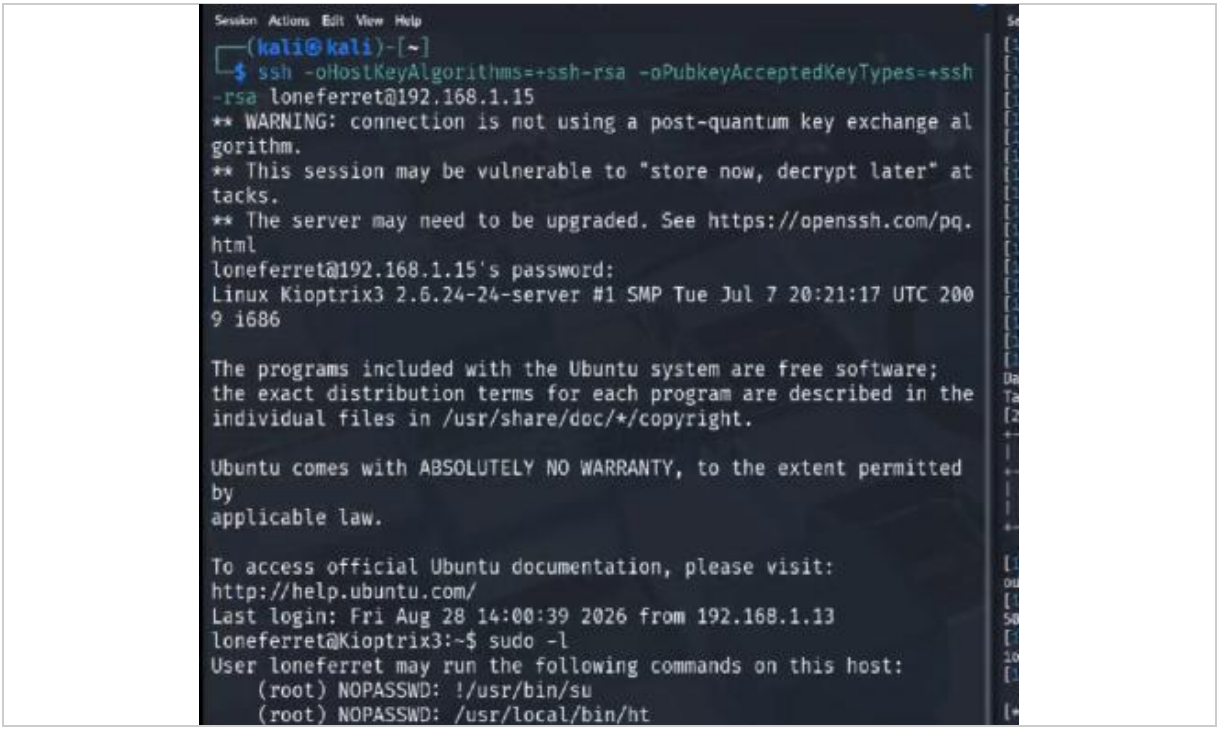


Figure 34. SSH login as 'loneferret' (password: starwars); sudo -l reveals NOPASSWD rights to run 'ht' (and an explicitly denied 'su').

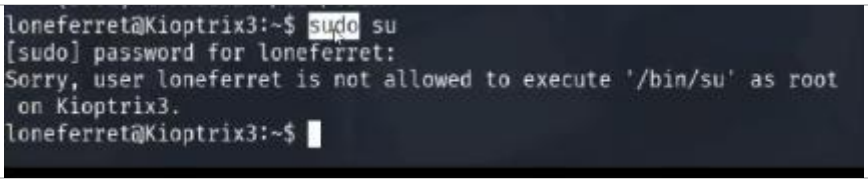


Figure 35. A direct 'sudo su' attempt is denied, confirming 'su' is blocked for this account.

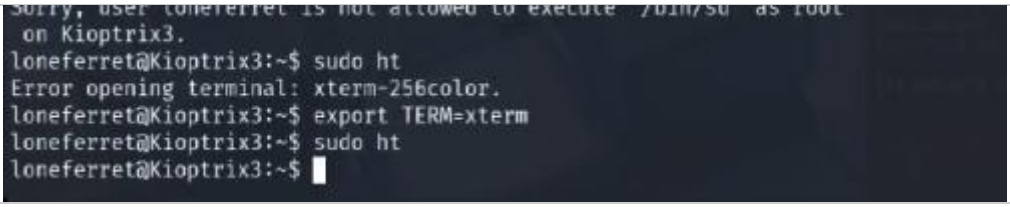


Figure 36. 'sudo ht' launches successfully after correcting the terminal type (export TERM=xterm).

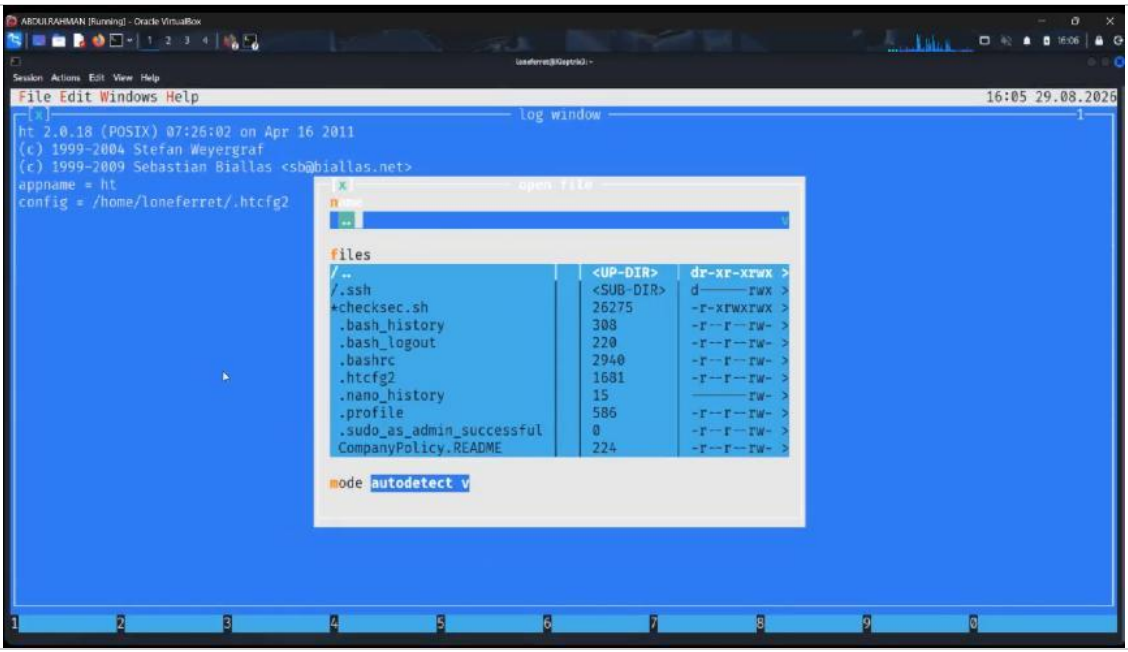


Figure 37. Running as root, ht's file browser can open any file on the system, including protected configuration files.

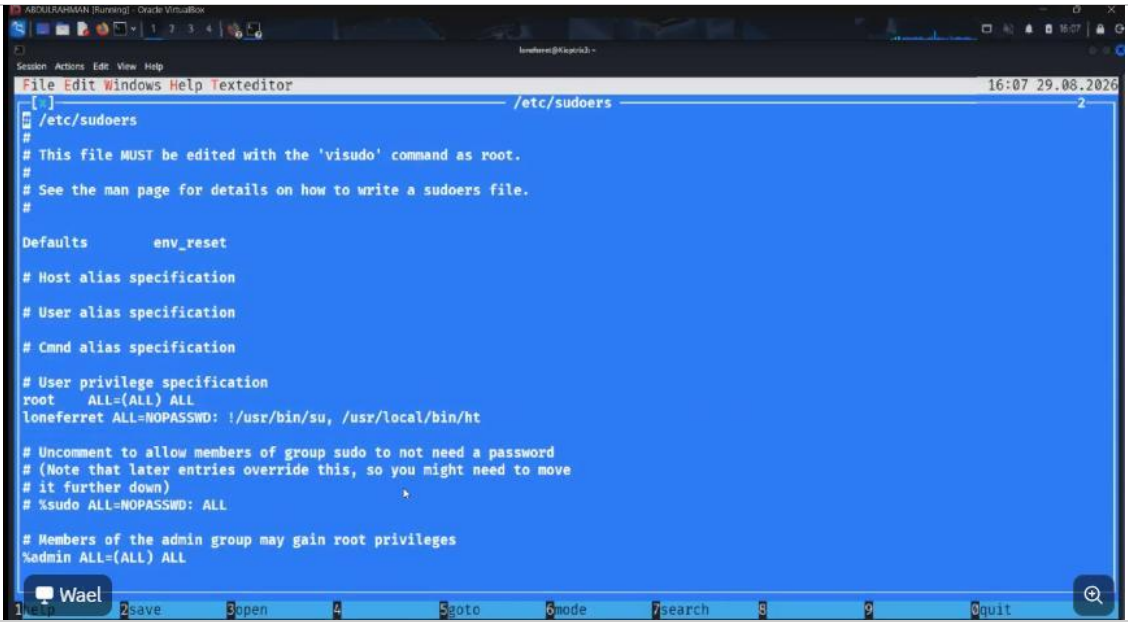


Figure 38. /etc/sudoers opened in ht, showing the original restricted entry for 'loneferret'.

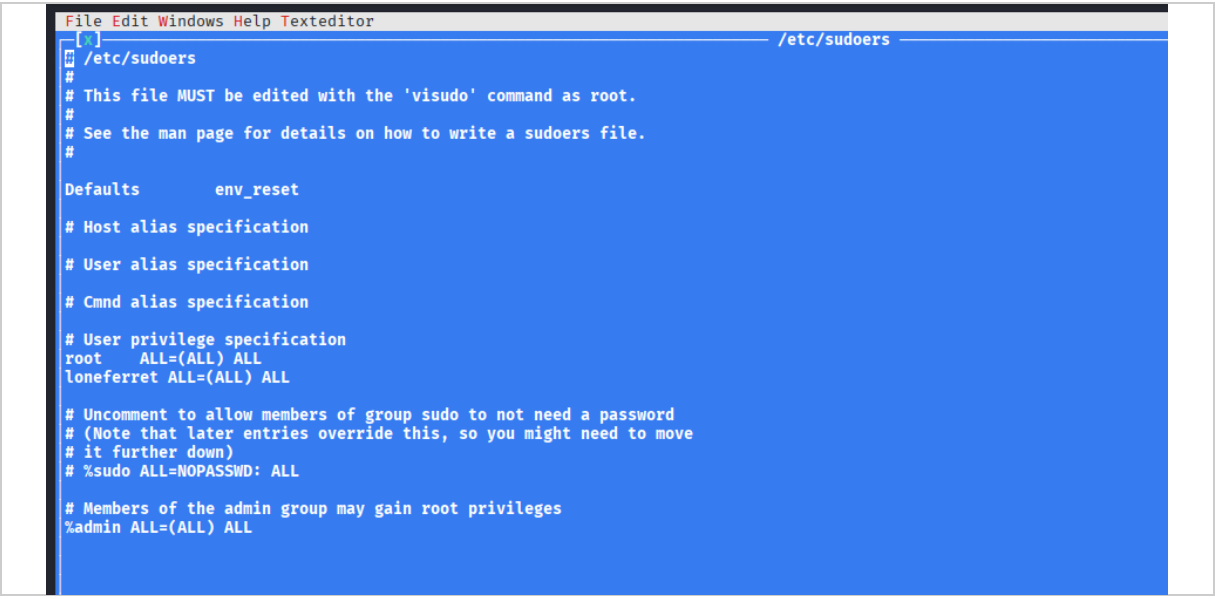


Figure 39. The 'loneferret' entry rewritten to ALL=(ALL) ALL, granting unrestricted sudo rights.

5.9 Proof of Full Compromise

With unrestricted sudo rights granted, 'sudo su' now succeeds, yielding a full root shell. Root access was confirmed via the host's built-in 'Congrats.txt' flag file and a final check of root-level identity and network commands.



Figure 40. Full root shell obtained; Congrats.txt read, confirming the machine was fully compromised.

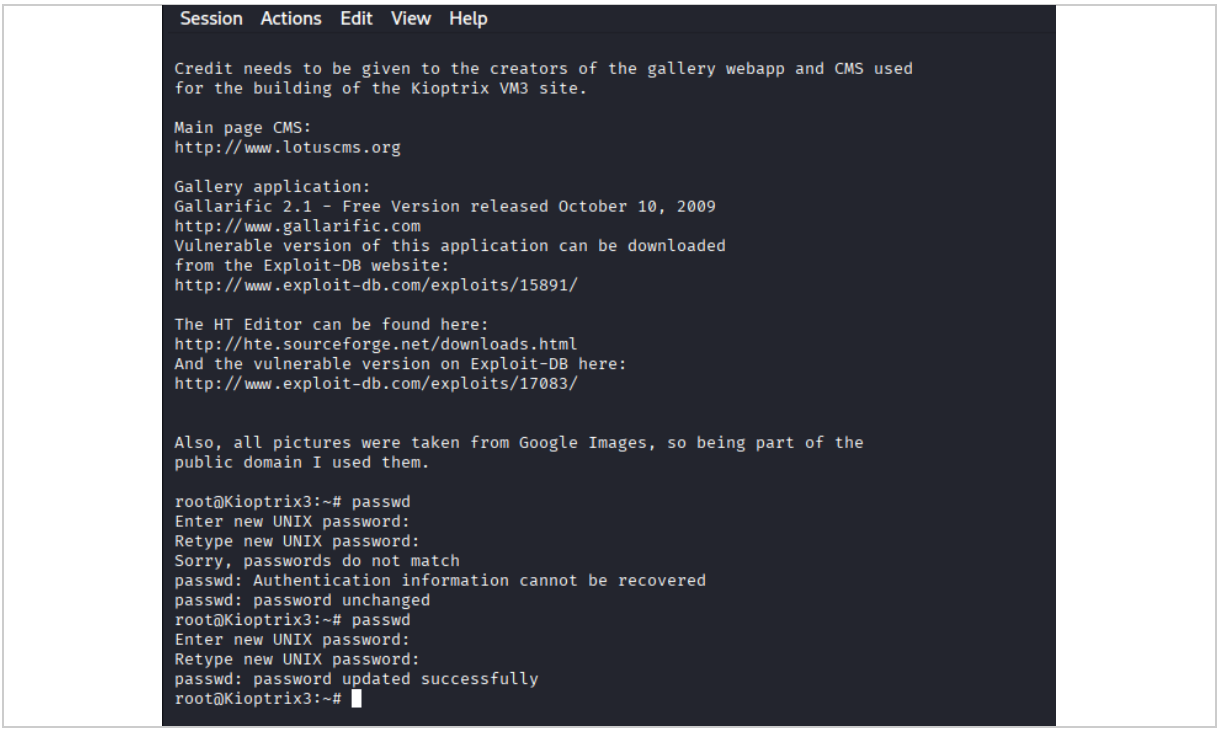


Figure 41. Challenge credits page and a successful root password change.

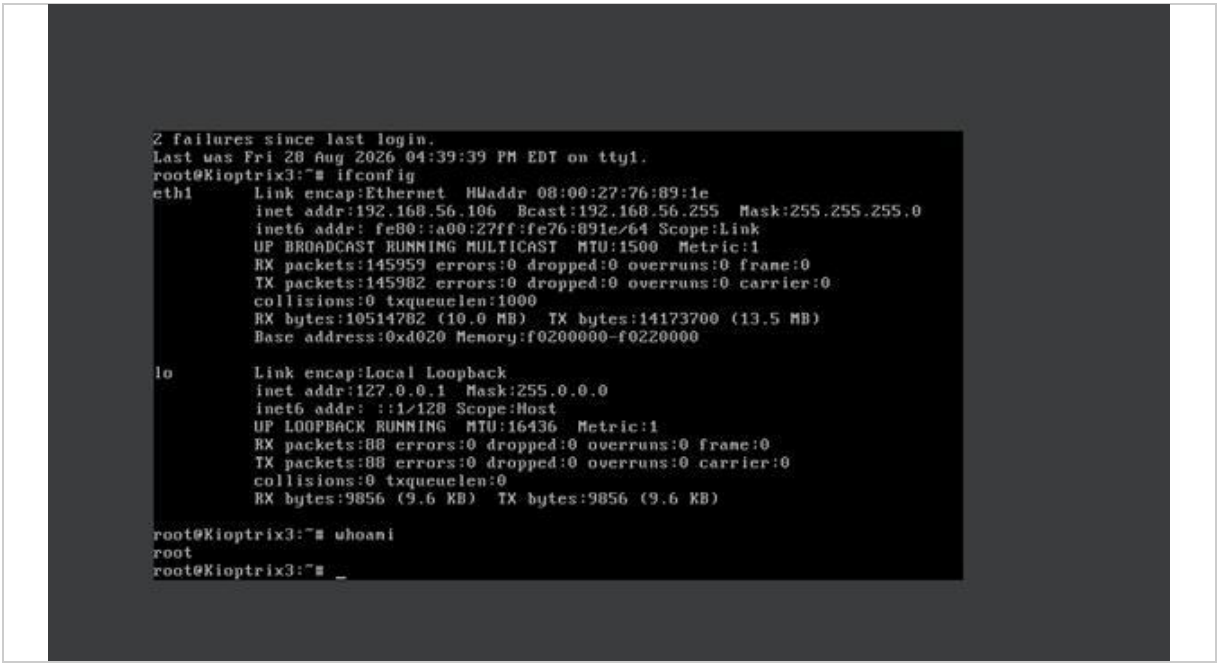


Figure 42. Final confirmation of full compromise — 'whoami' returns 'root' on the target host.

6. Findings Summary (Ordered by Severity)

The table below lists every issue identified during the assessment, ordered strictly by severity: Critical, then High, then Medium, then Low.

Finding	Severity	Description
SQL Injection in gallery.php (id / sort parameters)	Critical	The 'id' and 'sort' GET parameters of gallery.php were not sanitized, allowing boolean-based, error-based, time-based, and UNION-based SQL injection. Exploited with sqlmap and manual UNION payloads to enumerate databases, tables, and dump full credential tables.
Unauthenticated Remote Code Execution — LotusCMS 3.0 eval() injection	Critical	LotusCMS 3.0 passes user-controllable input into a PHP eval() call. Confirmed via a Parse error revealing core/lib/router.php, then weaponized with a public exploit script to run phpinfo() and system commands (id), and to obtain a full reverse shell as www-data.
Dangerous sudo entitlement — 'ht' editor abused to grant full root	Critical	User loneferret was permitted to run /usr/local/bin/ht as root via sudo with NOPASSWD. The full-featured ht editor was used to open and rewrite /etc/sudoers, changing loneferret's entry to ALL=(ALL) ALL, after which 'sudo su' produced an unrestricted root shell.
Full host compromise	Critical	Chaining the SQL injection, RCE, and sudo misconfiguration produced a complete path from unauthenticated network access to root-level control of the host.
Cleartext database credentials in application source	High	gallery/gconfig.php exposed the MySQL root credentials (root / fuckeyou) in plaintext, allowing full database access once the file was readable.
Weak, unsalted, crackable password hashes	High	MD5 hashes for local application users (dreg, loneferret) recovered via SQL injection and admin.dat were cracked in seconds via sqlmap's dictionary attack and hashcat (dreg = Mast3r, loneferret = starwars).
Credential reuse across application and OS accounts	High	The cracked application passwords for 'dreg' and 'loneferret' were both valid for SSH login to the underlying Linux host.
Outdated, end-of-life software stack	Medium	Nmap and phpinfo() identified EOL software: Ubuntu 8.04 (Hardy Heron), Apache 2.2.8, PHP 5.2.4, OpenSSH 4.7p1, and MySQL 5.0 — all with multiple known, unpatched CVEs.
World-writable web application directories	Medium	cache, core, data, modules, and style under the web root were world-writable (777), allowing tampering with application files and persistence.
Verbose error messages disclosing internal file paths	Low	A malformed eval() injection attempt returned a raw PHP parse error disclosing the full server path to core/lib/router.php, aiding further exploitation.

7. Attack Chain Summary

- 1. Reconnaissance identified an outdated Apache/PHP/OpenSSH stack on 192.168.56.106.
- 2. Enumeration revealed a Gallarific/LotusCMS-powered web application and an administration login page.
- 3. LotusCMS 3.0's unauthenticated eval() code-injection vulnerability was exploited (via script and manually) to obtain a www-data shell and confirm RCE.
- 4. Local enumeration surfaced a company policy note pointing to a privileged 'ht' tool, but no valid credentials yet.
- 5. SQL injection in gallery.php was discovered and exploited with sqlmap and manual UNION queries, dumping the 'dev_accounts' credential table.
- 6. The recovered MD5 hashes were cracked to plaintext: dreg = Mast3r, loneferret = starwars.
- 7. Both credentials were reused over SSH; 'dreg' had no sudo rights, but 'loneferret' matched the NOPASSWD 'ht' entitlement hinted at earlier.
- 8. 'sudo ht' was abused to directly rewrite /etc/sudoers, granting 'loneferret' full ALL=(ALL) ALL sudo rights.
- 9. 'sudo su' then produced a full root shell, confirmed via id, whoami, and the host's Congrats.txt flag file.

8. Recommendations

The following remediation actions are recommended, mapped to the findings in Section 6.

Area	Recommendation
Input validation	Use parameterized queries / prepared statements everywhere user input reaches SQL; never concatenate input into queries (fixes the gallery.php SQL injection).
Application security	Retire or upgrade LotusCMS/Gallarific; remove eval() calls that process user-controllable input, or strictly whitelist what can reach them.
Patch management	Upgrade the OS, Apache, PHP, MySQL and OpenSSH to currently supported versions and apply security updates on a regular cadence.
Sudo / SUID hardening	Remove NOPASSWD sudo rights to full-featured editors and shells (as demonstrated with 'ht'); such tools almost always provide a file-write or shell escape. Audit /etc/sudoers regularly.
Secrets management	Move database credentials out of source files (gconfig.php) into environment variables or a secrets manager; rotate the exposed MySQL root password immediately.
Password policy & hashing	Use bcrypt/argon2 with per-user salts instead of unsalted MD5; enforce strong, unique passwords to resist dictionary attacks (e.g. 'starwars', 'Mast3r').
Credential hygiene	Prohibit reuse of the same password between an application account and an OS-level account.
File system permissions	Remove world-writable permissions from application directories; apply least-privilege ownership for the web server user.
Error handling	Disable verbose PHP error/debug output in production (display_errors=Off) so internal paths and stack traces are never returned to clients.

Area	Recommendation
Monitoring & detection	Deploy host and network intrusion detection; log authentication, sudo usage, and database errors; alert on SQL-injection-pattern requests and unexpected sudoers changes.

9. Conclusion

This engagement successfully demonstrated a complete compromise of the Kioptrix Level 3 host via two independently exploitable web vulnerabilities — SQL injection and an unauthenticated RCE in LotusCMS — chained with plaintext credentials, weak password hashing, credential reuse, and an unsafe sudo configuration. Implementing the recommendations in Section 8 — particularly parameterizing all SQL queries, patching or retiring the vulnerable CMS, enforcing strong salted password hashing, and removing dangerous sudo entitlements — would have prevented or significantly hindered every stage of this attack chain.

This report was produced for educational purposes as part of the NTI Penetration Testing & Application Security track, using the intentionally vulnerable Kioptrix Level 3 training virtual machine.

Report prepared by

Abdulrahman Wael Nabil • Abdulrahman Hossam Shaat